arXiv:2011.10566v1 [cs.CV] 20 Nov 2020

Exploring Simple Siamese Representation Learning

Xinlei Chen Kaiming He
Facebook AI Research (FAIR)

Abstract

Siamese networks have become a common structure in various recent models for unsupervised visual representation learning. These models maximize the similarity between two augmentations of one image, subject to certain conditions for avoiding collapsing solutions. In this paper, we report surprising empirical results that ***simple Siamese*** *networks can learn meaningful representations even using* ***none*** *of the following: (i) negative sample pairs, (ii) large batches, (iii) momentum encoders. Our experiments show that collapsing solutions do exist for the loss and structure, but a stop-gradient operation plays an essential role in preventing collapsing. We provide a hypothesis on the implication of stop-gradient, and further show proof-of-concept experiments verifying it. Our "SimSiam" method achieves competitive results on ImageNet and downstream tasks. We hope this simple baseline will motivate people to rethink the roles of Siamese architectures for unsupervised representation learning. Code will be made available.*

1. Introduction

Recently there has been steady progress in un-/self-supervised representation learning, with encouraging results on multiple visual tasks (*e.g.*, [2, 17, 8, 15, 7]). Despite various original motivations, these methods generally involve certain forms of Siamese networks [4]. Siamese networks are weight-sharing neural networks applied on two or more inputs. They are natural tools for *comparing* (including but not limited to "*contrasting*") entities. Recent methods define the inputs as two augmentations of one image, and maximize the similarity subject to different conditions.

An undesired trivial solution to Siamese networks is all outputs "collapsing" to a constant. There have been several general strategies for preventing Siamese networks from collapsing. Contrastive learning [16], *e.g.*, instantiated in SimCLR [8], repulses different images (negative pairs) while attracting the same image's two views (positive pairs). The negative pairs preclude constant outputs from the solution space. Clustering [5] is another way of avoiding constant output, and SwAV [7] incorporates online clustering into Siamese networks. Beyond contrastive learning and clustering, BYOL [15] relies only on positive pairs but it does not collapse in case a momentum encoder is used.

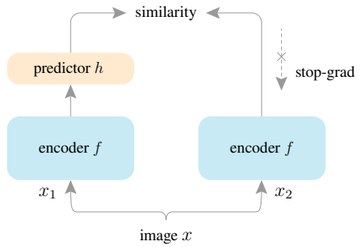


Figure 1. **SimSiam architecture**. Two augmented views of one image are processed by the same encoder network f (a backbone plus a projection MLP). Then a prediction MLP h is applied on one side, and a stop-gradient operation is applied on the other side. The model maximizes the similarity between both sides. It uses neither negative pairs nor a momentum encoder.

In this paper, we report that simple Siamese networks can work surprisingly well with *none* of the above strategies for preventing collapsing. Our model directly maximizes the similarity of one image's two views, using *neither* negative pairs *nor* a momentum encoder. It works with typical batch sizes and does not rely on large-batch training. We illustrate this "SimSiam" method in Figure 1.

Thanks to the conceptual simplicity, SimSiam can serve as a hub that relates several existing methods. In a nutshell, our method can be thought of as "BYOL *without* the momentum encoder". Unlike BYOL but like SimCLR and SwAV, our method directly shares the weights between the two branches, so it can also be thought of as "SimCLR *without* negative pairs", and "SwAV *without* online clustering". Interestingly, SimSiam is related to each method by removing one of its core components. Even so, SimSiam does not cause collapsing and can perform competitively.

We empirically show that collapsing solutions *do* exist, but a stop-gradient operation (Figure 1) is critical to prevent such solutions. The importance of stop-gradient suggests that there should be a different underlying optimization problem that is being solved. We hypothesize that there are implicitly two sets of variables, and SimSiam behaves like alternating between optimizing each set. We provide proof-of-concept experiments to verify this hypothesis.

探索简单孪生表征学习

Xinlei Chen Kaiming He
Facebook AI Research (FAIR)

摘要

孪生网络已成为近期多种无监督视觉表征学习模型的常见结构。这些模型在满足避免坍塌解的特定条件下，最大化同一图像两种增强的相似度。本文报告令人意外的实验结果：简单孪生网络即使完全不使用（*i*）负样本对、（*ii*）大批量、（*iii*）动量编码器，也能学习有意义的表征。实验表明，损失与结构确实存在坍塌解，但停止梯度操作对避免坍塌至关重要。我们提出关于停止梯度含义的假设，并通过概念验证实验进一步检验。*"SimSiam"*在 *ImageNet* 和下游任务上取得有竞争力的结果。希望这一简单基线促使人们重新思考孪生架构在无监督表征学习中的作用。代码将公开。

1 引言

近期无监督／自监督表征学习持续进步，在多种视觉任务上取得令人鼓舞的结果（如 [2, 17, 8, 15, 7]）。尽管最初动机不同，这些方法通常包含某种孪生网络 [4]。孪生网络是对两个或更多输入使用共享权重的神经网络，天然适合比较实体（包括但不限于"对比"）。近期方法将输入定义为同一图像的两种增强，在不同约束下最大化相似度。

孪生网络一种不理想的平凡解，是所有输出"坍塌"为常量。已有多种通用防坍塌策略。对比学习 [16]，（例如 SimCLR [8]，）推远不同图像的负样本对，同时拉近同一图像的两个正视图。负样本对使常量输出不属于解空间。聚类 [5]是另一种避免常量输出的方法；SwAV [7] 将在线聚类引入孪生网络。除了对比学习和聚类之外，BYOL [15] 只依赖正样本对；使用动量编码器时，它不发生坍塌。

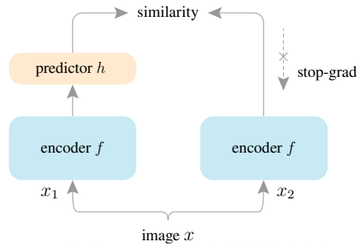


图1：SimSiam 架构。同一图像两个增强视图由同一编码器 f 处理（骨干加投影 MLP）。一侧应用预测 MLP h，另一侧停止梯度。模型最大化两侧相似度，既不用负样本对，也不用动量编码器。

本文发现，不采用上述任何防坍塌策略，简单孪生网络仍可表现出色。模型直接最大化同一图像两个视图的相似度，既不用负样本对，也不用动量编码器。普通批量即可有效训练，无需大批量。图1展示这一"SimSiam"方法。

概念上的简单性使 SimSiam 能作为连接已有方法的枢纽。本方法可视为"去掉动量编码器的 BYOL"。与 BYOL 不同，而与 SimCLR 和 SwAV 相同，我们直接在两个分支间共享权重；因此也可视为"去掉负样本对的 SimCLR"及"去掉在线聚类的 SwAV"。有趣的是，SimSiam 都通过移除这些方法的一个核心组件与之建立联系，却仍不坍塌，并具有竞争力。

实验表明坍塌解确实存在，但停止梯度操作（图1）对防止坍塌至关重要。其重要性暗示，实际求解的可能是另一个底层优化问题。我们假设其中隐含两组变量，SimSiam 的行为类似于交替优化各组变量，并给出概念验证实验检验这一假设。

Our simple baseline suggests that the Siamese architectures can be an essential reason for the common success of the related methods. Siamese networks can naturally introduce inductive biases for modeling invariance, as by definition "invariance" means that two observations of the same concept should produce the same outputs. Analogous to convolutions [25], which is a successful inductive bias via weight-sharing for modeling translation-invariance, the weight-sharing Siamese networks can model invariance w.r.t. more complicated transformations (*e.g.*, augmentations). We hope our exploration will motivate people to rethink the fundamental roles of Siamese architectures for unsupervised representation learning.

2. Related Work

Siamese networks. Siamese networks [4] are general models for comparing entities. Their applications include signature [4] and face [34] verification, tracking [3], one-shot learning [23], and others. In conventional use cases, the inputs to Siamese networks are from different images, and the comparability is determined by supervision.

Contrastive learning. The core idea of contrastive learning [16] is to attract the positive sample pairs and repulse the negative sample pairs. This methodology has been recently popularized for un-/self-supervised representation learning [36, 30, 20, 37, 21, 2, 35, 17, 29, 8, 9]. Simple and effective instantiations of contrastive learning have been developed using Siamese networks [37, 2, 17, 8, 9].

In practice, contrastive learning methods benefit from a large number of negative samples [36, 35, 17, 8]. These samples can be maintained in a memory bank [36]. In a Siamese network, MoCo [17] maintains a queue of negative samples and turns one branch into a momentum encoder to improve consistency of the queue. SimCLR [8] directly uses negative samples coexisting in the current batch, and it requires a large batch size to work well.

Clustering. Another category of methods for unsupervised representation learning are based on clustering [5, 6, 1, 7]. They alternate between clustering the representations and learning to predict the cluster assignment. SwAV [7] incorporates clustering into a Siamese network, by computing the assignment from one view and predicting it from another view. SwAV performs online clustering under a balanced partition constraint for each batch, which is solved by the Sinkhorn-Knopp transform [10].

While clustering-based methods do not define negative exemplars, the cluster centers can play as negative prototypes. Like contrastive learning, clustering-based methods require either a memory bank [5, 6, 1], large batches [7], or a queue [7] to provide enough samples for clustering.

BYOL. BYOL [15] directly predicts the output of one view from another view. It is a Siamese network in which one

Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
   x1, x2 = aug(x), aug(x) # random augmentation
   z1, z2 = f(x1), f(x2) # projections, n-by-d
   p1, p2 = h(z1), h(z2) # predictions, n-by-d

   L = D(p1, z2)/2 + D(p2, z1)/2 # loss

   L.backward() # back-propagate
   update(f, h) # SGD update

def D(p, z): # negative cosine similarity
   z = z.detach() # stop gradient

   p = normalize(p, dim=1) # l2-normalize
   z = normalize(z, dim=1) # l2-normalize
   return -(p*z).sum(dim=1).mean()
```

branch is a momentum encoder.[1] It is hypothesized in [15] that the momentum encoder is important for BYOL to avoid collapsing, and it reports failure results if removing the momentum encoder (0.3% accuracy, Table 5 in [15]).[2] Our empirical study challenges the *necessity* of the momentum encoder for preventing collapsing. We discover that the stop-gradient operation is critical. This discovery can be obscured with the usage of a momentum encoder, which is always accompanied with stop-gradient (as it is not updated by its parameters' gradients). While the moving-average behavior may improve accuracy with an appropriate momentum coefficient, our experiments show that it is not directly related to preventing collapsing.

3. Method

Our architecture (Figure 1) takes as input two randomly augmented views x_1 and x_2 from an image x. The two views are processed by an encoder network f consisting of a backbone (*e.g.*, ResNet [19]) and a projection MLP head [8]. The encoder f shares weights between the two views. A prediction MLP head [15], denoted as h, transforms the output of one view and matches it to the other view. Denoting the two output vectors as $p_1 \triangleq h(f(x_1))$ and $z_2 \triangleq f(x_2)$, we minimize their negative cosine similarity:

$$\mathcal{D}(p_1, z_2) = -\frac{p_1}{\|p_1\|_2} \cdot \frac{z_2}{\|z_2\|_2}, \tag{1}$$

where $\|\cdot\|_2$ is ℓ_2-norm. This is equivalent to the mean squared error of ℓ_2-normalized vectors [15], up to a scale

[1] MoCo [17] and BYOL [15] do not directly share the weights between the two branches, though in theory the momentum encoder should converge to the same status as the trainable encoder. We view these models as Siamese networks with "indirect" weight-sharing.

[2] In BYOL's arXiv v3 update, it reports 66.9% accuracy with 300-epoch pre-training when removing the momentum encoder and increasing the predictor's learning rate by 10×. Our work was done concurrently with this arXiv update. Our work studies this topic from different perspectives, with better results achieved.

这一简单基线表明，孪生架构可能是相关方法共同成功的重要原因。孪生网络天然可引入建模不变性的归纳偏置，因为"不变性"意味着对同一概念的两次观测应产生相同输出。卷积 [25], 通过权重共享建模平移不变性，是成功的归纳偏置；类似地，共享权重的孪生网络可建模更复杂变换（如增强）下的不变性。希望我们的探索促使人们重新思考孪生架构在无监督表征学习中的基本作用。

2 相关工作

孪生网络。孪生网络 [4] 是比较实体的通用模型，应用包括签名 [4] 、人脸 [34]验证、跟踪 [3], 、单样本学习 [23], 等。传统场景中，输入来自不同图像，可比较性由监督确定。

对比学习。核心思想 [16]是拉近正样本对、推远负样本对。该方法近期广泛用于无监督／自监督表征学习 [36, 30, 20, 37, 21, 2, 35, 17, 29, 8, 9]。使用孪生网络的简单、有效对比学习实现已经出现 [37, 2, 17, 8, 9]。

实践中，对比学习受益于大量负样本 [36, 35, 17, 8]。这些样本可保存在记忆库 [36]中。MoCo [17]在孪生网络中维护负样本队列，并将一个分支变为动量编码器，以改善队列一致性。SimCLR [8] 直接使用当前批次中的负样本，需要较大批量才能取得良好效果。

聚类。另一类无监督表征学习方法基于聚类 [5, 6, 1, 7], 交替执行表征聚类和学习预测聚类分配。SwAV [7] 将聚类引入孪生网络：由一个视图计算分配，再由另一视图预测。每个批次在均衡划分约束下执行在线聚类, 使用 Sinkhorn-Knopp 变换 [10]求解。

聚类方法虽不定义负样本，但聚类中心可充当负原型。类似对比学习，它们需要记忆库 [5, 6, 1], 、大批量 [7],或队列 [7]，为聚类提供足够样本。

BYOL。BYOL [15] 直接从一个视图预测另一视图的输出。它是孪生网络，其中一个

Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
   x1, x2 = aug(x), aug(x) # random augmentation
   z1, z2 = f(x1), f(x2) # projections, n-by-d
   p1, p2 = h(z1), h(z2) # predictions, n-by-d

   L = D(p1, z2)/2 + D(p2, z1)/2 # loss

   L.backward() # back-propagate
   update(f, h) # SGD update

def D(p, z): # negative cosine similarity
   z = z.detach() # stop gradient

   p = normalize(p, dim=1) # l2-normalize
   z = normalize(z, dim=1) # l2-normalize
   return -(p*z).sum(dim=1).mean()
```

分支是动量编码器。[1] 文献 [15]推测，动量编码器对 BYOL 避免坍塌很重要；移除它会失败（准确率0.3%, 见 [15]表5）。[2] 我们的实验质疑了动量编码器对于防止坍塌的必要性：停止梯度才是关键。这一发现可能被动量编码器掩盖，因为动量编码器总是同时停止梯度（其参数不通过自身梯度更新）。合适动量系数下的移动平均可能提高准确率，但实验表明，它与防止坍塌没有直接关系。

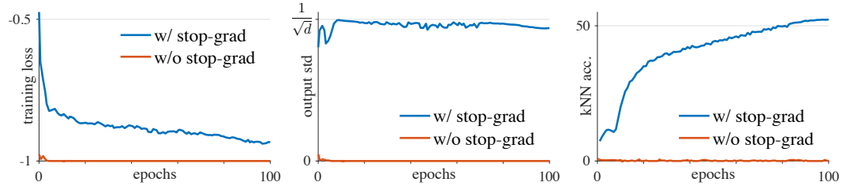
3 方法

我们的架构（图1）输入同一图像 x的两个随机增强视图 x_1 与 x_2 。两者经过编码器网络 f 处理；编码器包含骨干（如 ResNet [19]）及投影 MLP 头[8]。编码器 f 在两个视图之间共享权重。预测 MLP 头 [15], 记为 h，变换一个视图的输出，并使其匹配另一视图。将两个输出向量记为 $p_1 \triangleq h$（f（x_1）） 与 $z_2 \triangleq f$（x_2）, 最小化它们的负余弦相似度：

$$\mathcal{D}(p_1, z_2) = -\frac{p_1}{\|p_1\|_2} \cdot \frac{z_2}{\|z_2\|_2}, \tag{1}$$

其中 $\|\cdot\|_2$ 为 ℓ_2范数。除比例因子外，它等价于 ℓ_2归一化向量的均方误差 [15], 。

[1] MoCo [17] 与 BYOL [15] 没有直接在两个分支间共享权重，但理论上动量编码器应收敛至可训练编码器的同一状态。我们将其视为"间接"共享权重的孪生网络。[2]BYOL 的 arXiv 第3版报告，移除动量编码器并将预测器学习率提高 10×，预训练300轮得到66.9%的准确率。我们的研究与该更新同期完成，从不同角度研究了这一问题，并取得更好结果。



	acc. (%)
w/ stop-grad	67.7±0.1
w/o stop-grad	0.1

Figure 2. **SimSiam with *vs*. without stop-gradient**. **Left plot**: training loss. Without stop-gradient it degenerates immediately. **Middle plot**: the per-channel std of the ℓ_2-normalized output, plotted as the averaged std over all channels. **Right plot**: validation accuracy of a kNN classifier [36] as a monitor of progress. **Table**: ImageNet linear evaluation ("w/ stop-grad" is mean±std over 5 trials).

of 2. Following [15], we define a symmetrized loss as:

$$\mathcal{L} = \frac{1}{2}\mathcal{D}(p_1, z_2) + \frac{1}{2}\mathcal{D}(p_2, z_1). \tag{2}$$

This is defined for each image, and the total loss is averaged over all images. Its minimum possible value is -1.

An important component for our method to work is a stop-gradient (`stopgrad`) operation (Figure 1). We implement it by modifying (1) as:

$$\mathcal{D}(p_1, \texttt{stopgrad}(z_2)). \tag{3}$$

This means that z_2 is treated as a constant in this term. Similarly, the form in (2) is implemented as:

$$\mathcal{L} = \frac{1}{2}\mathcal{D}(p_1, \texttt{stopgrad}(z_2)) + \frac{1}{2}\mathcal{D}(p_2, \texttt{stopgrad}(z_1)). \tag{4}$$

Here the encoder on x_2 receives no gradient from z_2 in the first term, but it receives gradients from p_2 in the second term (and vice versa for x_1).

The pseudo-code of SimSiam is in Algorithm 1.

Baseline settings. Unless specified, our explorations use the following settings for unsupervised pre-training:

- *Optimizer*. We use SGD for pre-training. Our method does *not* require a large-batch optimizer such as LARS [38] (unlike [8, 15, 7]). We use a learning rate of $lr \times \text{BatchSize}/256$ (linear scaling [14]), with a base $lr = 0.05$. The learning rate has a cosine decay schedule [27, 8]. The weight decay is 0.0001 and the SGD momentum is 0.9.

  The batch size is 512 by default, which is friendly to typical 8-GPU implementations. Other batch sizes also work well (Sec. 4.3). We use batch normalization (BN) [22] synchronized across devices, following [8, 15, 7].

- *Projection MLP*. The projection MLP (in f) has BN applied to each fully-connected (fc) layer, including its output fc. Its output fc has no ReLU. The hidden fc is 2048-d. This MLP has 3 layers.

- *Prediction MLP*. The prediction MLP (h) has BN applied to its hidden fc layers. Its output fc does not have BN (ablation in Sec. 4.4) or ReLU. This MLP has 2 layers. The dimension of h's input and output (z and p) is $d = 2048$, and h's hidden layer's dimension is 512, making h a bottleneck structure (ablation in supplement).

We use ResNet-50 [19] as the default backbone. Other implementation details are in supplement. We perform 100-epoch pre-training in ablation experiments.

Experimental setup. We do unsupervised pre-training on the 1000-class ImageNet training set [11] without using labels. The quality of the pre-trained representations is evaluated by training a supervised linear classifier on frozen representations in the training set, and then testing it in the validation set, which is a common protocol. The implementation details of linear classification are in supplement.

4. Empirical Study

In this section we empirically study the SimSiam behaviors. We pay special attention to what may contribute to the model's non-collapsing solutions.

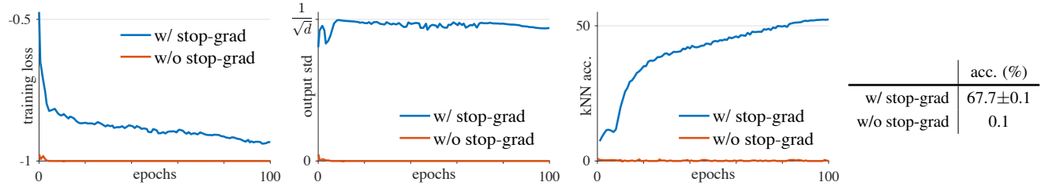
4.1. Stop-gradient

Figure 2 presents a comparison on "with *vs*. without stop-gradient". The architectures and all hyper-parameters are kept unchanged, and stop-gradient is the only difference.

Figure 2 (left) shows the training loss. *Without* stop-gradient, the optimizer quickly finds a degenerated solution and reaches the minimum possible loss of -1. To show that the degeneration is caused by collapsing, we study the standard deviation (std) of the ℓ_2-normalized output $z/\|z\|_2$. If the outputs collapse to a constant vector, their std over all samples should be zero for each channel. This can be observed from the red curve in Figure 2 (middle).

As a comparison, if the output z has a zero-mean isotropic Gaussian distribution, we can show that the std of $z/\|z\|_2$ is $\frac{1}{\sqrt{d}}$.[3] The blue curve in Figure 2 (middle) shows

[3]Here is an informal derivation: denote $z/\|z\|_2$ as z', that is, $z'_i = z_i/(\sum_{j=1}^{d} z_j^2)^{\frac{1}{2}}$ for the i-th channel. If z_j is subject to an i.i.d Gaussian distribution: $z_j \sim \mathcal{N}(0,1), \forall j$, then $z'_i \approx z_i/d^{\frac{1}{2}}$ and $\text{std}[z'_i] \approx 1/d^{\frac{1}{2}}$.



	acc. (%)
w/ stop-grad	67.7±0.1
w/o stop-grad	0.1

图2：SimSiam 使用与不使用停止梯度的比较。左图：训练损失，不停止梯度会立即退化。中图：ℓ_2归一化输出的逐通道标准差，对所有通道取平均。右图：作为训练进度监测器的 kNN 分类器 [36] 验证准确率。表格：ImageNet 线性评估（"w/ stop-grad"为5次试验的均值±标准差）。

为2。按照 [15]，定义对称损失：

$$\mathcal{L} = \frac{1}{2}\mathcal{D}(p_1, z_2) + \frac{1}{2}\mathcal{D}(p_2, z_1). \tag{2}$$

该损失对每张图像定义，再对全部图像求平均；可能的最小值为 -1。

本方法有效的关键组件是停止梯度（stopgrad，图1）。将式（1）改写为：

$$\mathcal{D}(p_1, \texttt{stopgrad}(z_2)). \tag{3}$$

这意味着在该项中 z_2 被视为常量。同样，式（2）实现为：

$$\mathcal{L} = \frac{1}{2}\mathcal{D}(p_1, \texttt{stopgrad}(z_2)) + \frac{1}{2}\mathcal{D}(p_2, \texttt{stopgrad}(z_1)). \tag{4}$$

第一项中，x_2 上的编码器不接收来自 z_2 的梯度，但第二项中接收来自 p_2 的梯度（x_1同理）。

SimSiam 伪代码见算法1。

基线设置。除非另有说明，无监督预训练探索采用以下设置：

• 优化器。预训练使用 SGD。本方法无需 LARS[38] 这类大批量优化器（不同于 [8, 15, 7]）。学习率为 $lr \times \text{BatchSize}/256$（线性缩放 [14]），基础 $lr = 0.05$。学习率采用余弦衰减[27, 8]，权重衰减0.0001，SGD 动量0.9。

默认批量512，适合常见8块 GPU 实现。其他批量也有效（第4.3节）。按照 [8, 15, 7]，使用跨设备同步的批归一化（BN）[22]。

• 投影 MLP。投影 MLP（位于 f）的每个全连接（fc）层均使用 BN，包括输出层。输出层没有 ReLU；隐藏层为2048维。MLP 共3层。

• 预测 MLP。预测 MLP（h）的隐藏全连接层使用 BN，输出层没有 BN（第4.4节进行消融）或 ReLU。该 MLP 共2层。h 的输入与输出（z 、 p）维数为 d =2048， h的隐藏层为512维，因此 h具有瓶颈结构（补充材料中消融）。

默认骨干为 ResNet-50 [19]，其他实现细节见补充材料。消融实验预训练100轮。

实验设置。在1000类 ImageNet 训练集 [11] 上进行无监督预训练，不使用标签。用训练集上的冻结表征训练有监督线性分类器，再在验证集测试，以评估预训练表征质量。这是通用评估协议，线性分类实现细节见补充材料。

4 实验研究

本节通过实验研究 SimSiam 的行为，特别关注哪些因素可能使模型得到不坍塌的解。

4.1 停止梯度

图2比较有无停止梯度。架构和全部超参数不变，唯一区别是是否停止梯度。

图2左图展示训练损失。不停止梯度时，优化器迅速找到退化解，并达到可能的最小损失 -1。为证明退化来自坍塌，我们研究 ℓ_2归一化输出 $z/\|z\|_2$的标准差（std）。若输出坍塌为常量向量，每个通道在所有样本上的标准差都应为零，图2中图的红线显示了这一现象。

作为比较，若输出 z 服从零均值各向同性高斯分布，可证明$z/\|z\|_2$ 的标准差为 $\frac{1}{\sqrt{d}}$。[3] 图2中图蓝线显示

[3]非严格推导如下：将 $z/\|z\|_2$ 记为 z'，即第 i个通道的 $z'_i = z_i/(\sum_{j=1}^{d} z_j^2)^{\frac{1}{2}}$。若$z_j$服从独立同分布高斯分布 $z_j \sim \mathcal{N}(0, 1)$，$\forall j$，则 $z'_i \approx z_i/d^{\frac{1}{2}}$及 $\text{std}[z'_i] \approx 1/d^{\frac{1}{2}}$。

	pred. MLP h	acc. (%)
baseline	lr with cosine decay	67.7
(a)	no pred. MLP	0.1
(b)	fixed random init.	1.5
(c)	lr not decayed	68.1

Table 1. **Effect of prediction MLP** (ImageNet linear evaluation accuracy with 100-epoch pre-training). In all these variants, we use the same schedule for the encoder f (lr with cosine decay).

that with stop-gradient, the std value is near $\frac{1}{\sqrt{d}}$. This indicates that the outputs do not collapse, and they are scattered on the unit hypersphere.

Figure 2 (right) plots the validation accuracy of a k-nearest-neighbor (kNN) classifier [36]. This kNN classifier can serve as a monitor of the progress. With stop-gradient, the kNN monitor shows a steadily improving accuracy.

The linear evaluation result is in the table in Figure 2. SimSiam achieves a nontrivial accuracy of 67.7%. This result is reasonably stable as shown by the std of 5 trials. Solely removing stop-gradient, the accuracy becomes 0.1%, which is the chance-level guess in ImageNet.

Discussion. Our experiments show that *there exist collapsing solutions*. The collapse can be observed by the minimum possible loss and the constant outputs.[4] The existence of the collapsing solutions implies that it is *insufficient* for our method to prevent collapsing *solely* by the architecture designs (*e.g.*, predictor, BN, ℓ_2-norm). In our comparison, all these architecture designs are kept unchanged, but they do not prevent collapsing if stop-gradient is removed.

The introduction of stop-gradient implies that there should be another optimization problem that is being solved underlying. We propose a hypothesis in Sec. 5.

4.2. Predictor

In Table 1 we study the predictor MLP's effect.

The model does not work if removing h (Table 1a), *i.e.*, h is the identity mapping. Actually, this observation can be expected if the symmetric loss (4) is used. Now the loss is $\frac{1}{2}\mathcal{D}(z_1, \texttt{stopgrad}(z_2)) + \frac{1}{2}\mathcal{D}(z_2, \texttt{stopgrad}(z_1))$. Its gradient has the same direction as the gradient of $\mathcal{D}(z_1, z_2)$, with the magnitude scaled by $1/2$. In this case, using stop-gradient is equivalent to removing stop-gradient and scaling the loss by 1/2. Collapsing is observed (Table 1a).

We note that this derivation on the gradient direction is valid only for the symmetrized loss. But we have observed that the *asymmetric* variant (3) also fails if removing h, while it can work if h is kept (Sec. 4.6). These experiments suggest that h is helpful for our model.

If h is fixed as random initialization, our model does not work either (Table 1b). However, this failure is *not* about collapsing. The training does not converge, and the loss remains high. The predictor h should be trained to adapt to the representations.

We also find that h with a constant lr (without decay) can work well and produce even better results than the baseline (Table 1c). A possible explanation is that h should adapt to the latest representations, so it is not necessary to force it converge (by reducing lr) before the representations are sufficiently trained. In many variants of our model, we have observed that h with a constant lr provides slightly better results. We use this form in the following subsections.

[4] We note that a chance-level accuracy (0.1%) is not sufficient to indicate collapsing. A model with a diverging loss, which is another pattern of failure, may also exhibit a chance-level accuracy.

batch size	64	128	256	512	1024	2048	4096
acc. (%)	66.1	67.3	68.1	68.1	68.0	67.9	64.0

Table 2. **Effect of batch sizes** (ImageNet linear evaluation accuracy with 100-epoch pre-training).

case	proj. MLP's BN hidden	proj. MLP's BN output	pred. MLP's BN hidden	pred. MLP's BN output	acc. (%)
(a) none	-	-	-	-	34.6
(b) hidden-only	✓	-	✓	-	67.4
(c) default	✓	✓	✓	-	68.1
(d) all	✓	✓	✓	✓	unstable

Table 3. **Effect of batch normalization on MLP heads** (ImageNet linear evaluation accuracy with 100-epoch pre-training).

4.3. Batch Size

Table 2 reports the results with a batch size from 64 to 4096. When the batch size changes, we use the same linear scaling rule ($lr \times$BatchSize/256) [14] with base $lr = 0.05$. We use 10 epochs of warm-up [14] for batch sizes $\geq$1024. Note that we keep using the same SGD optimizer (rather than LARS [38]) for all batch sizes studied.

Our method works reasonably well over this wide range of batch sizes. Even a batch size of 128 or 64 performs decently, with a drop of 0.8% or 2.0% in accuracy. The results are similarly good when the batch size is from 256 to 2048, and the differences are at the level of random variations.

This behavior of SimSiam is noticeably different from SimCLR [8] and SwAV [7]. All three methods are Siamese networks with direct weight-sharing, but SimCLR and SwAV both require a large batch (*e.g.*, 4096) to work well.

We also note that the standard SGD optimizer does not work well when the batch is too large (even in supervised learning [14, 38]), and our result is lower with a 4096 batch. We expect a specialized optimizer (*e.g.*, LARS [38]) will help in this case. However, our results show that a specialized optimizer is *not* necessary for preventing collapsing.

4.4. Batch Normalization

Table 3 compares the configurations of BN on the MLP heads. In Table 3a we remove all BN layers in the MLP heads (10-epoch warmup [14] is used specifically for this

	预测 MLP h	准确率（%）
基线	lr 及余弦衰减	67.7
(a)	无预测 MLP	0.1
(b)	固定随机初始化	1.5
(c)	lr 不衰减	68.1

表1：预测 MLP 的影响（预训练100轮后的 ImageNet 线性评估准确率）。所有变体的编码器 f 均采用相同日程（lr 及余弦衰减）。

停止梯度时，标准差接近 $\frac{1}{\sqrt{d}}$，说明输出没有坍塌，而是散布在单位超球面上。

图2右图绘制 k 近邻（kNN）分类器 [36]的验证准确率，可用于监测训练进度。停止梯度时，kNN 准确率稳步提高。

图2表格展示线性评估。SimSiam 达到非平凡的67.7%准确率，5次试验的标准差表明结果较稳定。仅去掉停止梯度，准确率变成0.1%，相当于ImageNet 的随机猜测。

讨论。实验表明坍塌解确实存在，可由可能的最小损失及常量输出观察到。[4] 这意味着，仅靠架构设计（如预测器、BN、 ℓ_2范数）不足以避免坍塌。在比较中，这些设计保持不变；但去掉停止梯度后，它们都无法阻止坍塌。

停止梯度暗示，底层正在求解另一个优化问题。第5节提出假设。

4.2 预测器

表1研究预测 MLP 的影响。

去掉 h （即h为恒等映射）后模型无法工作，见表1a。使用对称损失（4）时，这一现象其实可以预期。此时损失为 $\frac{1}{2}\mathcal{D}$（z_1，stopgrad（z_2））$+\frac{1}{2}\mathcal{D}$（z_2，stopgrad（z_1））。它的梯度与 $\mathcal{D}$（z_1，z_2）的梯度方向相同，幅值乘以 1/2。此时，停止梯度等价于去掉停止梯度并将损失乘1/2，因此仍出现坍塌（表1a）。

上述梯度方向推导仅对对称损失成立。但实验发现，不对称变体（3）去掉 h也失败，保留 h 则有效（第4.6节）。这些实验说明 h对模型有益。

将 h 固定为随机初始化，模型也无法工作（表1b）。但这一失败不是

[4] 随机水平准确率（0.1%）不足以证明坍塌。损失发散的模型也可能产生随机水平准确率，而这是另一种失败模式。

批量大小	64	128	256	512	1024	2048	4096
准确率（%）	66.1	67.3	68.1	68.1	68.0	67.9	64.0

表2：批量大小的影响（预训练100轮后的 ImageNet 线性评估准确率）。

case	投影 MLP 的 BN 隐藏层	投影 MLP 的 BN 输出	预测 MLP 的 BN 隐藏层	预测 MLP 的 BN 输出	准确率（%）
(a) none	-	-	-	-	34.6
(b) 仅隐藏层	✓	-	✓	-	67.4
(c) 默认	✓	✓	✓	-	68.1
(d) all	✓	✓	✓	✓	不稳定

表3：MLP 头批归一化的影响（预训练100轮后的 ImageNet 线性评估准确率）。

坍塌。训练不收敛，损失保持较高。预测器 h 应通过训练适应表征。

我们还发现， h 使用恒定 lr （不衰减）也能有效训练，且比基线更好（表1c）。一种解释是， h 应适应最新表征，因此没必要在表征充分训练前，就通过降低 lr强迫其收敛。在许多变体中， h 使用恒定 lr 结果略好；后续小节采用这一形式。

4.3 批量大小

表2报告批量64至4096的结果。改变批量时，使用相同线性缩放规则（$lr \times$BatchSize/256） [14] 及基础 $lr = 0.05$。批量 $\geq$1024时，使用10轮预热 [14] 。所有批量均使用同一 SGD 优化器，而非 LARS [38]。

本方法在这一广泛批量范围内均表现良好。即使批量128或64也不错，准确率分别下降0.8或2.0个百分点。批量256至2048时结果相近，差别处于随机波动范围内。

SimSiam 的这一行为明显不同于 SimCLR [8]与 SwAV [7]。三者都是直接共享权重的孪生网络，但后两者都需要大批量（如4096）才能表现良好。

批量过大时，标准 SGD 也表现不佳（即使在有监督学习中 [14, 38]），批量4096的结果因此较低。专用优化器（如 LARS [38]）可能有帮助。但结果表明，专用优化器不是防止坍塌的必要条件。

4.4 批归一化

表3比较 MLP 头的 BN 配置。表3a移除其中全部 BN 层（专门为该配置使用10轮预热 [14] ，

entry). This variant does *not* cause collapse, although the accuracy is low (34.6%). The low accuracy is likely because of optimization difficulty. Adding BN to the hidden layers (Table 3b) increases accuracy to 67.4%.

Further adding BN to the output of the *projection* MLP (*i.e.*, the output of f) boosts accuracy to 68.1% (Table 3c), which is our default configuration. In this entry, we also find that the learnable affine transformation (scale and offset [22]) in f's output BN is not necessary, and disabling it leads to a comparable accuracy of 68.2%.

Adding BN to the output of the prediction MLP h does not work well (Table 3d). We find that this is not about collapsing. The training is unstable and the loss oscillates.

In summary, we observe that BN is helpful for optimization when used appropriately, which is similar to BN's behavior in other *supervised* learning scenarios. But we have seen no evidence that BN helps to prevent collapsing: actually, the comparison in Sec. 4.1 (Figure 2) has exactly the same BN configuration for both entries, but the model collapses if stop-gradient is not used.

4.5. Similarity Function

Besides the cosine similarity function (1), our method also works with cross-entropy similarity. We modify $\mathcal{D}$ as: $\mathcal{D}(p_1, z_2) = -\texttt{softmax}(z_2) \cdot \log \texttt{softmax}(p_1)$. Here the softmax function is along the channel dimension. The output of softmax can be thought of as the probabilities of belonging to each of d pseudo-categories.

We simply replace the cosine similarity with the cross-entropy similarity, and symmetrize it using (4). All hyper-parameters and architectures are unchanged, though they may be suboptimal for this variant. Here is the comparison:

	cosine	cross-entropy
acc. (%)	68.1	63.2

The cross-entropy variant can converge to a reasonable result without collapsing. This suggests that the collapsing prevention behavior is not just about the cosine similarity.

This variant helps to set up a connection to SwAV [7], which we discuss in Sec. 6.2.

4.6. Symmetrization

Thus far our experiments have been based on the symmetrized loss (4). We observe that SimSiam's behavior of preventing collapsing does not depend on symmetrization. We compare with the *asymmetric* variant (3) as follows:

	sym.	asym.	asym. 2×
acc. (%)	68.1	64.8	67.3

The asymmetric variant achieves reasonable results. Symmetrization is helpful for boosting accuracy, but it is not related to collapse prevention. Symmetrization makes one more prediction for each image, and we may roughly compensate for this by sampling two pairs for each image in the asymmetric version ("2×"). It makes the gap smaller.

4.7. Summary

We have empirically shown that in a variety of settings, SimSiam can produce meaningful results without collapsing. The optimizer (batch size), batch normalization, similarity function, and symmetrization may affect accuracy, but we have seen *no* evidence that they are related to collapse prevention. It is mainly the stop-gradient operation that plays an essential role.

5. Hypothesis

We discuss a hypothesis on what is implicitly optimized by SimSiam, with proof-of-concept experiments provided.

5.1. Formulation

Our hypothesis is that SimSiam is an implementation of an Expectation-Maximization (EM) like algorithm. It implicitly involves two sets of variables, and solves two underlying sub-problems. The presence of stop-gradient is the consequence of introducing the extra set of variables.

We consider a loss function of the following form:

$$\mathcal{L}(\theta, \eta) = \mathbb{E}_{x,\mathcal{T}}\Big[\big\|\mathcal{F}_\theta(\mathcal{T}(x)) - \eta_x\big\|_2^2\Big]. \tag{5}$$

$\mathcal{F}$ is a network parameterized by θ. $\mathcal{T}$ is the augmentation. x is an image. The expectation $\mathbb{E}[\cdot]$ is over the distribution of images and augmentations. For the ease of analysis, here we use the mean squared error $\|\cdot\|_2^2$, which is equivalent to the cosine similarity if the vectors are ℓ_2-normalized. We do not consider the predictor yet and will discuss it later.

In (5), we have introduced *another set of variables* which we denote as η. The size of η is proportional to the number of images. Intuitively, η_x is the representation of the image x, and the subscript $_x$ means using the image index to access a sub-vector of η. η is *not necessarily* the output of a network; it is the argument of an optimization problem.

With this formulation, we consider solving:

$$\min_{\theta,\eta} \mathcal{L}(\theta, \eta). \tag{6}$$

Here the problem is w.r.t. both θ and η. This formulation is analogous to k-means clustering [28]. The variable θ is analogous to the clustering centers: it is the learnable parameters of an encoder. The variable η_x is analogous to the assignment vector of the sample x (a one-hot vector in k-means): it is the representation of x.

Also analogous to k-means, the problem in (6) can be solved by an alternating algorithm, fixing one set of variables and solving for the other set. Formally, we can alternate between solving these two subproblems:

$$\theta^t \leftarrow \arg\min_{\theta} \mathcal{L}(\theta, \eta^{t-1}) \tag{7}$$

$$\eta^t \leftarrow \arg\min_{\eta} \mathcal{L}(\theta^t, \eta) \tag{8}$$

Here t is the index of alternation and "$\leftarrow$" means assigning.

配置）。该变体不坍塌，但准确率较低（34.6%），可能是优化困难所致。给隐藏层加入 BN（表3b），准确率升至67.4%。

再给投影 MLP 输出（即 f的输出）加入 BN，准确率升至68.1%（表3c），这是默认配置。此配置中，f输出 BN 的可学习仿射变换（缩放及偏移 [22]）并非必要；禁用后得到相近的68.2%。

给预测 MLP h 的输出加 BN 效果不佳（表3d）。这不是坍塌问题，而是训练不稳定、损失振荡。

总之，适当使用 BN 有助于优化，与其他有监督场景类似。但没有证据表明 BN 有助于防止坍塌：第4.1节（图2）的两项比较采用完全相同的 BN 配置，不停止梯度的模型仍然坍塌。

4.5 相似性函数

除余弦相似性函数（1），本方法也适用于交叉熵相似性。将 $\mathcal{D}$ 改为：$\mathcal{D}$（p_1，z_2）$=-$softmax（z_2）$\cdot$ log softmax（p_1）。softmax 沿通道维计算，其输出可视为属于 d 个伪类别的概率。

直接将余弦相似性替换为交叉熵相似性，并用式（4）对称化。所有超参数和架构不变，尽管对该变体可能并非最优。比较如下：

	余弦	交叉熵
准确率（%）	68.1	63.2

交叉熵变体也可收敛到合理结果而不坍塌，表明防坍塌并非仅由余弦相似性决定。

该变体帮助建立与 SwAV [7],的联系，第6.2节讨论。

4.6 对称化

此前实验均使用对称损失（4）。我们发现，SimSiam 防坍塌不依赖对称化，与不对称变体（3）的比较如下：

	sym.	不对称	不对称 2×
准确率（%）	68.1	64.8	67.3

不对称变体结果合理。对称化有助于提高准确率，但与防止坍塌无关。对称化使每张图像多做一次预测；在不对称版本中，每张图像采样两对（“2×”），可大致补偿这一点并缩小差距。

4.7 小结

实验表明，多种设置下 SimSiam 都能产生有意义的结果而不坍塌。优化器（批量大小）、批归一化、相似性函数及对称化可能影响准确率，但没有证据显示它们与避免坍塌有关；主要是停止梯度起关键作用。

5 假设

我们讨论 SimSiam 隐含优化内容的假设，并给出概念验证实验。

5.1 问题形式

我们假设 SimSiam 是类似期望最大化（EM）算法的一种实现：隐含两组变量，求解两个底层子问题。停止梯度是引入额外变量组的结果。

考虑如下形式的损失函数：

$$\mathcal{L}(\theta, \eta) = \mathbb{E}_{x,\mathcal{T}}\Big[\big\|\mathcal{F}_\theta(\mathcal{T}(x)) - \eta_x\big\|_2^2\Big]. \tag{5}$$

$\mathcal{F}$ 是由 θ参数化的网络，$\mathcal{T}$为增强，x 为图像。期望 $\mathbb{E}[\cdot]$ 遍历图像与增强的分布。为便于分析，这里用均方误差 $\|\cdot\|_2^2$；向量经过 ℓ_2归一化时，它等价于余弦相似度。暂不考虑预测器，稍后讨论。

式（5）引入另一组变量，记为 η。η 的规模与图像数成正比。直观地，η_x 是图像x的表征；下标$_x$ 表示用图像索引访问 η的子向量。η 不一定是网络输出，而是优化问题的自变量。

在这一形式下，考虑求解：

$$\min_{\theta,\eta} \mathcal{L}(\theta, \eta). \tag{6}$$

此处问题同时涉及 θ 和 η，类似于 k-means 聚类 [28]。变量 θ 类似聚类中心，是编码器的可学习参数；变量 η_x 类似样本 x 的分配向量（k-means 中的独热向量），即 x的表征。

同样类似 k-means，式（6）可用交替算法求解：固定一组变量，求解另一组。形式上，可以交替求解以下两个子问题：

$$\theta^t \leftarrow \arg\min_{\theta} \mathcal{L}(\theta, \eta^{t-1}) \tag{7}$$

$$\eta^t \leftarrow \arg\min_{\eta} \mathcal{L}(\theta^t, \eta) \tag{8}$$

t 为交替索引，“$\leftarrow$”表示赋值。

Solving for θ. One can use SGD to solve the sub-problem (7). *The stop-gradient operation is a natural consequence*, because the gradient does not back-propagate to η^{t-1} which is a constant in this subproblem.

Solving for η. The sub-problem (8) can be solved independently for each η_x. Now the problem is to minimize: $\mathbb{E}_{\mathcal{T}}\left[\|\mathcal{F}_{\theta^t}(\mathcal{T}(x)) - \eta_x\|_2^2\right]$ for each image x, noting that the expectation is over the distribution of augmentation $\mathcal{T}$. Due to the mean squared error,[5] it is easy to solve it by:

$$\eta_x^t \leftarrow \mathbb{E}_{\mathcal{T}}\Big[\mathcal{F}_{\theta^t}(\mathcal{T}(x))\Big]. \tag{9}$$

This indicates that η_x is assigned with the *average* representation of x over the distribution of augmentation.

One-step alternation. SimSiam can be approximated by one-step alternation between (7) and (8). First, we approximate (9) by sampling the augmentation only *once*, denoted as $\mathcal{T}'$, and ignoring $\mathbb{E}_{\mathcal{T}}[\cdot]$:

$$\eta_x^t \leftarrow \mathcal{F}_{\theta^t}(\mathcal{T}'(x)). \tag{10}$$

Inserting it into the sub-problem (7), we have:

$$\theta^{t+1} \leftarrow \arg\min_{\theta} \mathbb{E}_{x,\mathcal{T}}\Big[\big\|\mathcal{F}_{\theta}(\mathcal{T}(x)) - \mathcal{F}_{\theta^t}(\mathcal{T}'(x))\big\|_2^2\Big]. \tag{11}$$

Now θ^t is a constant in this sub-problem, and $\mathcal{T}'$ implies another view due to its random nature. This formulation exhibits the Siamese architecture. Second, if we implement (11) by *reducing* the loss with one SGD step, then we can approach the SimSiam algorithm: a Siamese network naturally with stop-gradient applied.

Predictor. Our above analysis does not involve the predictor h. We further assume that h is helpful in our method because of the approximation due to (10).

By definition, the predictor h is expected to minimize: $\mathbb{E}_z\left[\|h(z_1) - z_2\|_2^2\right]$. The optimal solution to h should satisfy: $h(z_1) = \mathbb{E}_z[z_2] = \mathbb{E}_{\mathcal{T}}\big[f(\mathcal{T}(x))\big]$ for any image x. This term is similar to the one in (9). In our approximation in (10), the expectation $\mathbb{E}_{\mathcal{T}}[\cdot]$ is ignored. The usage of h may fill this gap. In practice, it would be unrealistic to actually compute the expectation $\mathbb{E}_{\mathcal{T}}$. But it may be possible for a neural network (*e.g.*, the preditor h) to learn to predict the expectation, while the sampling of $\mathcal{T}$ is implicitly distributed across multiple epochs.

[5] If we use the cosine similarity, we can approximately solve it by ℓ_2-normalizing $\mathcal{F}$'s output and η_x.

Symmetrization. Our hypothesis does not involve symmetrization. Symmetrization is like denser sampling $\mathcal{T}$ in (11). Actually, the SGD optimizer computes the empirical expectation of $\mathbb{E}_{x,\mathcal{T}}[\cdot]$ by sampling a batch of images and one pair of augmentations ($\mathcal{T}_1$, $\mathcal{T}_2$). In principle, the empirical expectation should be more precise with denser sampling. Symmetrization supplies an extra pair ($\mathcal{T}_2$, $\mathcal{T}_1$). This explains that symmetrization is not necessary for our method to work, yet it is able to improve accuracy, as we have observed in Sec. 4.6.

5.2. Proof of concept

We design a series of proof-of-concept experiments that stem from our hypothesis. They are methods different with SimSiam, and they are designed to verify our hypothesis.

Multi-step alternation. We have hypothesized that the SimSiam algorithm is like alternating between (7) and (8), with an interval of one step of SGD update. Under this hypothesis, it is likely for our formulation to work if the interval has multiple steps of SGD.

In this variant, we treat t in (7) and (8) as the index of an outer loop; and the sub-problem in (7) is updated by an inner loop of k SGD steps. In each alternation, we pre-compute the η_x required for all k SGD steps using (10) and cache them in memory. Then we perform k SGD steps to update θ. We use the same architecture and hyper-parameters as SimSiam. The comparison is as follows:

	1-step	10-step	100-step	1-epoch
acc. (%)	68.1	68.7	68.9	67.0

Here, "1-step" is equivalent to SimSiam, and "1-epoch" denotes the k steps required for one epoch. All multi-step variants work well. The 10-/100-step variants even achieve *better* results than SimSiam, though at the cost of extra precomputation. This experiment suggests that the alternating optimization is a valid formulation, and SimSiam is a special case of it.

Expectation over augmentations. The usage of the predictor h is presumably because the expectation $\mathbb{E}_{\mathcal{T}}[\cdot]$ in (9) is ignored. We consider another way to approximate this expectation, in which we find h is not needed.

In this variant, we do not update η_x directly by the assignment (10); instead, we maintain a moving-average: $\eta_x^t \leftarrow m * \eta_x^{t-1} + (1-m) * \mathcal{F}_{\theta^t}(\mathcal{T}'(x))$, where m is a momentum coefficient (0.8 here). This computation is similar to maintaining the *memory bank* as in [36]. This moving-average provides an approximated expectation of multiple views. This variant has 55.0% accuracy *without* the predictor h. As a comparison, it fails completely if we remove h but do not maintain the moving average (as shown in Table 1a). This proof-of-concept experiment supports that the usage of predictor h is related to approximating $\mathbb{E}_{\mathcal{T}}[\cdot]$.

求解 θ。可用 SGD 求解子问题（7）。停止梯度是自然结果，因为该子问题中 η^{t-1} 为常量，梯度不向其反向传播。

求解 η。子问题（8）可对每个 η_x独立求解。对每张图像 x，问题是最小化$\mathbb{E}_{\mathcal{T}}\left[\|\mathcal{F}_{\theta^t}（\mathcal{T}（x））-\eta_x\|_2^2\right]$，其中期望遍历增强 $\mathcal{T}$的分布。由于使用均方误差[5]，易得如下解：

$$\eta_x^t \leftarrow \mathbb{E}_{\mathcal{T}}\Big[\mathcal{F}_{\theta^t}(\mathcal{T}(x))\Big]. \tag{9}$$

这意味着， η_x 被赋值为 x在增强分布上的平均表征。

单步交替。SimSiam 可近似为式（7）和（8）之间的单步交替。首先仅采样一次增强，记为 $\mathcal{T}'$，忽略 $\mathbb{E}_{\mathcal{T}}[\cdot]$, 以近似式（9）：

$$\eta_x^t \leftarrow \mathcal{F}_{\theta^t}(\mathcal{T}'(x)). \tag{10}$$

代入子问题（7），得到：

$$\theta^{t+1} \leftarrow \arg\min_{\theta} \mathbb{E}_{x,\mathcal{T}}\Big[\big\|\mathcal{F}_{\theta}(\mathcal{T}(x)) - \mathcal{F}_{\theta^t}(\mathcal{T}'(x))\big\|_2^2\Big]. \tag{11}$$

在该子问题中， θ^t 为常量， $\mathcal{T}'$ 的随机性意味着另一个视图。因此该形式体现孪生架构。其次，若用一个 SGD 步降低损失来实现式（11），就接近 Si
mSiam：自然地使用停止梯度的孪生网络。

预测器。上述分析不包含预测器 h。我们进一步假设，由于式（10）的近似， h 对方法有益。

根据定义，预测器 h 应最小化：$\mathbb{E}_z\left[\|h（z_1）- z_2\|_2^2\right]$。对任意图像 x， h 的最优解应满足： $h（z_1）=\mathbb{E}_z[z_2]=\mathbb{E}_{\mathcal{T}}\big[f（\mathcal{T}（x））\big]$。这一项类似于式（9）中的项。在式（10）的近似中，我们忽略了期望 $\mathbb{E}_{\mathcal{T}}[\cdot]$；使用 h 可能弥补这一缺口。实践中，真正计算期望 $\mathbb{E}_{\mathcal{T}}$ 并不现实。但神经网络（如预测器 h）可能学会预测该期望，而对 $\mathcal{T}$的采样隐含地分布在多个训练轮次中。

[5] 若使用余弦相似性，可通过对 $\mathcal{F}$的输出及 η_x做 ℓ_2归一化近似求解。

对称化。我们的假设不涉及对称化。对称化类似于在式（11）中更密集地采样 $\mathcal{T}$。SGD 通过采样一批图像及一对增强（$\mathcal{T}_1$， $\mathcal{T}_2$），计算$\mathbb{E}_x$、$\tau[\cdot]$ 的经验期望。原则上，采样越密集，经验期望应越准确。对称化提供额外的一对（$\mathcal{T}_2$， $\mathcal{T}_1$）。这解释了为何对称化不是本方法有效的必要条件，却能提高准确率，与第4.6节观察一致。

5.2 概念验证

基于假设设计一系列概念验证实验。它们不同于 SimSiam，目的是验证假设。

多步交替。我们假设 SimSiam 在式（7）和（8）之间交替，每隔一个 SGD 更新步切换。若该假设成立，将间隔改为多个 SGD 步也应有效。

该变体将式（7）、（8）中的 t 视为外循环索引，式（7）的子问题由包含 k 个 SGD 步的内循环更新。每次交替，先用式（10）预计算这 k 个 SGD 步所需的 η_x 并缓存在内存，再执行 k 个 SGD 步更新 θ。架构和超参数与 SimSiam 相同，比较如下：

	1步	10步	100步	1轮
准确率（%）	68.1	68.7	68.9	67.0

“1-step”等价于 SimSiam， “1-epoch”表示一轮所需的 k 步。所有多步变体均有效；10步和100步变体甚至优于 SimSiam，但增加了预计算开销。这表明交替优化是合理的形式，SimSiam 是其特例。

增强上的期望。使用预测器 h 可能是因为忽略了式（9）中的期望 $\mathbb{E}_{\mathcal{T}}[\cdot]$。考虑另一种近似期望的方式，此时我们发现 h 并非必需。

该变体不通过赋值式（10）直接更新 η_x，而是维护移动平均：$\eta_x^t \leftarrow m * \eta_x^{t-1} + (1-m) * \mathcal{F}_{\theta^t}（\mathcal{T}'（x））$，其中 m 为动量系数（此处为0.8）。该计算类似于 [36]中维护记忆库的方式。移动平均提供了多个视图期望的近似。不使用预测器 h时，该变体准确率为55.0%。相比之下，如果去掉 h且不维护移动平均，方法会完全失败（表1a）。这一概念验证实验支持了预测器 h 与近似 $\mathbb{E}_{\mathcal{T}}[\cdot]$有关的观点。

method	batch size	negative pairs	momentum encoder	100 ep	200 ep	400 ep	800 ep
SimCLR (repro.+)	4096	✓		66.5	68.3	69.8	70.4
MoCo v2 (repro.+)	**256**	✓	✓	67.4	69.9	71.0	72.2
BYOL (repro.)	4096		✓	66.5	**70.6**	**73.2**	**74.3**
SwAV (repro.+)	4096			66.5	69.1	70.7	71.8
SimSiam	**256**			**68.1**	70.0	70.8	71.3

Table 4. **Comparisons on ImageNet linear classification**. All are based on **ResNet-50** pre-trained with **two 224×224 views**. Evaluation is on a single crop. All competitors are from our reproduction, and "+" denotes *improved* reproduction *vs*. original papers (see supplement).

pre-train	VOC 07 detection			VOC 07+12 detection			COCO detection			COCO instance seg.		
	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}	AP^{mask}_{50}	AP^{mask}	AP^{mask}_{75}
scratch	35.9	16.8	13.0	60.2	33.8	33.1	44.0	26.4	27.8	46.9	29.3	30.8
ImageNet supervised	74.4	42.4	42.7	81.3	53.5	58.8	58.2	38.2	41.2	54.7	33.3	35.2
SimCLR (repro.+)	75.9	46.8	50.1	81.8	55.5	61.4	57.7	37.9	40.9	54.6	33.3	35.3
MoCo v2 (repro.+)	**77.1**	**48.5**	**52.5**	**82.3**	**57.0**	**63.3**	**58.8**	**39.2**	**42.5**	**55.5**	**34.3**	**36.6**
BYOL (repro.)	**77.1**	47.0	49.9	81.4	55.3	61.1	57.8	37.9	40.9	54.3	33.2	35.0
SwAV (repro.+)	75.5	46.5	49.6	81.5	55.4	61.4	57.6	37.6	40.3	54.2	33.1	35.1
SimSiam, base	75.5	47.0	50.2	**82.0**	56.4	62.8	57.5	37.9	40.9	54.2	33.2	35.2
SimSiam, optimal	**77.3**	**48.5**	**52.5**	**82.4**	**57.0**	**63.7**	**59.3**	**39.2**	**42.1**	**56.0**	**34.4**	**36.7**

Table 5. **Transfer Learning**. All unsupervised methods are based on 200-epoch pre-training in ImageNet. *VOC 07 detection*: Faster R-CNN [32] fine-tuned in VOC 2007 trainval, evaluated in VOC 2007 test; *VOC 07+12 detection*: Faster R-CNN fine-tuned in VOC 2007 trainval + 2012 train, evaluated in VOC 2007 test; *COCO detection* and *COCO instance segmentation*: Mask R-CNN [18] (1× schedule) fine-tuned in COCO 2017 train, evaluated in COCO 2017 val. All Faster/Mask R-CNN models are with the C4-backbone [13]. All VOC results are the average over 5 trials. **Bold entries** are within 0.5 below the best.

5.3. Discussion

Our hypothesis is about what the optimization problem can be. It does not explain why collapsing is prevented. We point out that SimSiam and its variants' non-collapsing behavior still remains as an empirical observation.

Here we briefly discuss our understanding on this open question. The alternating optimization provides a different trajectory, and the trajectory depends on the initialization. It is *unlikely* that the initialized η, which is the output of a randomly initialized network, would be a constant. Starting from this initialization, it may be difficult for the alternating optimizer to approach a constant η_x for all x, because the method does *not* compute the gradients w.r.t. η jointly for all x. The optimizer seeks another trajectory (Figure 2 left), in which the outputs are scattered (Figure 2 middle).

6. Comparisons

6.1. Result Comparisons

ImageNet. We compare with the state-of-the-art frameworks in Table 4 on ImageNet linear evaluation. For fair comparisons, all competitors are based on our reproduction, and "+" denotes *improved* reproduction *vs*. the original papers (see supplement). For each individual method, we follow the hyper-parameter and augmentation recipes in its original paper.[6] All entries are based on a standard ResNet-50, with two 224×224 views used during pre-training.

[6]In our BYOL reproduction, the 100, 200(400), 800-epoch recipes follow the 100, 300, 1000-epoch recipes in [15]: lr is {0.45, 0.3, 0.2}, wd is {1e-6, 1e-6, 1.5e-6}, and momentum coefficient is {0.99, 0.99, 0.996}.

Table 4 shows the results and the main properties of the methods. SimSiam is trained with a batch size of 256, using neither negative samples nor a momentum encoder. Despite it simplicity, SimSiam achieves competitive results. It has the highest accuracy among all methods under 100-epoch pre-training, though its gain of training longer is smaller. It has better results than SimCLR in all cases.

Transfer Learning. In Table 5 we compare the representation quality by transferring them to other tasks, including VOC [12] object detection and COCO [26] object detection and instance segmentation. We fine-tune the pre-trained models end-to-end in the target datasets. We use the public codebase from MoCo [17] for all entries, and search the fine-tuning learning rate for each individual method. All methods are based on 200-epoch pre-training in ImageNet using our reproduction.

Table 5 shows that SimSiam's representations are *transferable* beyond the ImageNet task. It is competitive among these leading methods. The "base" SimSiam in Table 5 uses the baseline pre-training recipe as in our ImageNet experiments. We find that another recipe of $lr = 0.5$ and $wd = 1e\text{-}5$ (with similar ImageNet accuracy) can produce better results in all tasks (Table 5, "SimSiam, optimal").

We emphasize that *all these methods are highly successful for transfer learning*—in Table 5, they can surpass or be on par with the ImageNet *supervised* pre-training counterparts in all tasks. Despite many design differences, a common structure of these methods is the *Siamese network*. This comparison suggests that the Siamese structure is a core factor for their general success.

方法	批量 size	负样本对	动量编码器	100轮	200轮	400轮	800轮
SimCLR（复现+）	4096	✓		66.5	68.3	69.8	70.4
MoCo v2（复现+）	**256**	✓	✓	67.4	69.9	71.0	72.2
BYOL（复现）	4096		✓	66.5	**70.6**	**73.2**	**74.3**
SwAV（复现+）	4096			66.5	69.1	70.7	71.8
SimSiam	**256**			**68.1**	70.0	70.8	71.3

表4：ImageNet 线性分类比较。全部采用 ResNet-50，以两个 **224×224** 视图预训练，单次裁剪评估。对比方法均由我们复现；"+"表示优于原论文的改进复现（见补充材料）。

预训练	VOC 07检测			VOC 07+12 检测			COCO 检测			COCO 实例分割		
	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}	AP^{mask}_{50}	AP^{mask}	AP^{mask}_{75}
从零训练	35.9	16.8	13.0	60.2	33.8	33.1	44.0	26.4	27.8	46.9	29.3	30.8
ImageNet 有监督	74.4	42.4	42.7	81.3	53.5	58.8	58.2	38.2	41.2	54.7	33.3	35.2
SimCLR（复现+）	75.9	46.8	50.1	81.8	55.5	61.4	57.7	37.9	40.9	54.6	33.3	35.3
MoCo v2（复现+）	**77.1**	**48.5**	**52.5**	**82.3**	**57.0**	**63.3**	**58.8**	**39.2**	**42.5**	**55.5**	**34.3**	**36.6**
BYOL（复现）	**77.1**	47.0	49.9	81.4	55.3	61.1	57.8	37.9	40.9	54.3	33.2	35.0
SwAV（复现+）	75.5	46.5	49.6	81.5	55.4	61.4	57.6	37.6	40.3	54.2	33.1	35.1
SimSiam，base	75.5	47.0	50.2	**82.0**	56.4	62.8	57.5	37.9	40.9	54.2	33.2	35.2
SimSiam，最优配置	**77.3**	**48.5**	**52.5**	**82.4**	**57.0**	**63.7**	**59.3**	**39.2**	**42.1**	**56.0**	**34.4**	**36.7**

表5：迁移学习。所有无监督方法均在 ImageNet 上预训练200轮。VOC 07检测：Faster R-CNN [32] 在 VOC 2007 trainval 上微调，在 VOC 2007 test 上评估；VOC *07+12* 检测：Faster R-CNN 在 VOC 2007 trainval + 2012 train 上微调，在 VOC 2007 test 上评估；COCO 检测与实例分割：Mask R-CNN [18] 采用1× 训练日程，在 COCO 2017 train 上微调，在 COCO 2017 val 上评估。全部 Faster／Mask R-CNN 使用 C4 骨干 [13]。VOC 结果为5次试验平均值。与最佳值相差不超过0.5的条目以粗体表示。

5.3 讨论

我们的假设讨论优化问题可能是什么，并未解释为何避免坍塌。SimSiam 及其变体不坍塌的行为，仍只是实验观察。

这里简要讨论对这一开放问题的理解。交替优化产生不同轨迹，且轨迹依赖初始化。随机初始化网络的输出 η不太可能为常量。由此出发，交替优化器可能难以使所有 x的 η_x 趋于同一常量，因为该方法并不针对所有 x联合计算关于 η 的梯度。优化器寻找另一条轨迹（图2左图），其输出分散（图2中图）。

6 比较

6.1 结果比较

ImageNet。表4比较最先进框架的 ImageNet 线性评估。为公平起见，对比方法均由我们复现；"+"表示优于原论文的改进复现（见补充材料）。每种方法均遵循原论文的超参数与增强配置。[6] 全部使用标准 ResNet-50，预训练时使用两个 224×224视图。

[6]复现 BYOL 时，100、200（400）、800轮配置分别遵循 [15]中100、300、1000轮配置：lr 为 {0.45、0.3、0.2}，wd 为{1e-6、1e-6、1.5e-6}，动量系数为 {0.99、0.99、0.996}。

表4展示结果和方法主要属性。SimSiam 批量为256，既无负样本，也无动量编码器。虽然简单，结果仍具竞争力。在100轮预训练下，其准确率是所有方法中最高的，但延长训练带来的增益较小。所有情况下均优于 SimCLR。

迁移学习。表5通过迁移到 VOC [12] 目标检测，以及 COCO [26] 目标检测和实例分割等任务，比较表征质量。预训练模型在目标数据集上端到端微调。所有方法都使用 MoCo [17]的公开代码库，分别搜索各方法的微调学习率。它们均基于我们复现的 ImageNet 200轮预训练。

表5表明，SimSiam 的表征可以迁移到 ImageNet 之外的任务，在这些领先方法中具有竞争力。表5的"base"采用与 ImageNet 实验相同的基线预训练配置。另一组 $lr = 0.5$和 $wd = 1e\text{-}5$配置具有相近 ImageNet 准确率，却在所有任务上更好（表5"SimSiam, optimal"）。

这些方法的迁移学习都非常成功：表5中，它们在所有任务上都超过或相当于 ImageNet 有监督预训练。尽管设计差异很多，共同结构是孪生网络。这说明孪生结构是其普遍成功的核心因素。

6.2. Methodology Comparisons

Beyond accuracy, we also compare the methodologies of these Siamese architectures. Our method plays as a hub to connect these methods. Figure 3 abstracts these methods. The "encoder" subsumes all layers that can be shared between both branches (*e.g.*, backbone, projection MLP [8], prototypes [7]). The components in red are those missing in SimSiam. We discuss the relations next.

Relation to SimCLR [8]. SimCLR relies on negative samples ("dissimilarity") to prevent collapsing. SimSiam can be thought of as "SimCLR without negatives".

To have a more thorough comparison, we append the prediction MLP h and stop-gradient to SimCLR.[7] Here is the ablation on our SimCLR reproduction:

SimCLR	w/ predictor	w/ pred. & stop-grad
66.5	66.4	66.0

Neither the stop-gradient nor the extra predictor is necessary or helpful for SimCLR. As we have analyzed in Sec. 5, the introduction of the stop-gradient and extra predictor is presumably a consequence of another underlying optimization problem. It is different from the contrastive learning problem, so these extra components may not be helpful.

Relation to SwAV [7]. SimSiam is conceptually analogous to "SwAV without online clustering". We build up this connection by recasting a few components in SwAV. (i) The shared prototype layer in SwAV can be absorbed into the Siamese encoder. (ii) The prototypes were weight-normalized outside of gradient propagation in [7]; we instead implement by full gradient computation [33].[8] (iii) The similarity function in SwAV is cross-entropy. With these abstractions, a highly simplified SwAV illustration is shown in Figure 3.

SwAV applies the Sinkhorn-Knopp (SK) transform [10] on the target branch (which is also symmetrized [7]). The SK transform is derived from online clustering [7]: it is the outcome of clustering the current batch subject to a balanced partition constraint. The balanced partition can avoid collapsing. Our method does not involve this transform.

We study the effect of the prediction MLP h and stop-gradient on SwAV. Note that SwAV applies stop-gradient on the SK transform, so we ablate by removing it. Here is the comparison on our SwAV reproduction:

SwAV	w/ predictor	remove stop-grad
66.5	65.2	NaN

Adding the predictor does not help either. Removing stop-gradient (so the model is trained end-to-end) leads to divergence. As a clustering-based method, SwAV is inherently

[7] We append the extra predictor to one branch and stop-gradient to the other branch, and symmetrize this by swapping.

[8] This modification produces similar results as original SwAV, but it can enable end-to-end propagation in our ablation.

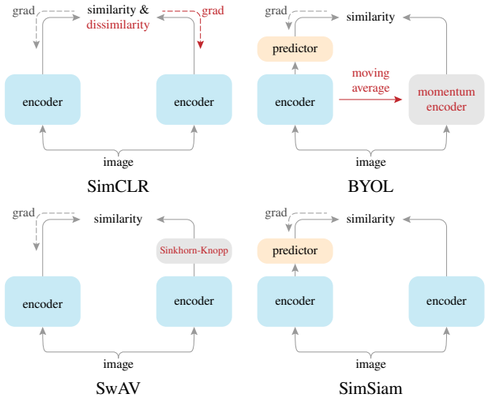


Figure 3. **Comparison on Siamese architectures**. The encoder includes all layers that can be shared between both branches. The dash lines indicate the gradient propagation flow. In BYOL, SwAV, and SimSiam, the lack of a dash line implies stop-gradient, and their symmetrization is not illustrated for simplicity. The components in red are those missing in SimSiam.

an alternating formulation [7]. This may explain why stop-gradient should not be removed from SwAV.

Relation to BYOL [15]. Our method can be thought of as "BYOL without the momentum encoder", subject to many implementation differences. The momentum encoder may be beneficial for accuracy (Table 4), but it is not necessary for preventing collapsing. Given our hypothesis in Sec. 5, the η sub-problem (8) can be solved by other optimizers, *e.g.*, a gradient-based one. This may lead to a temporally smoother update on η. Although not directly related, the momentum encoder also produces a smoother version of η. We believe that other optimizers for solving (8) are also plausible, which can be a future research problem.

7. Conclusion

We have explored Siamese networks with simple designs. The competitiveness of our minimalist method suggests that the Siamese shape of the recent methods can be a core reason for their effectiveness. Siamese networks are natural and effective tools for modeling invariance, which is a focus of representation learning. We hope our study will attract the community's attention to the fundamental role of Siamese networks in representation learning.

References

[1] Yuki Markus Asano, Christian Rupprecht, and Andrea Vedaldi. Self-labelling via simultaneous clustering and representation learning. *arXiv:1911.05371*, 2019.

6.2 方法原理比较

除准确率外，还比较这些孪生架构的方法原理。本方法是连接它们的枢纽。图3进行抽象："编码器"包括可由两个分支共享的全部层（如骨干、投影MLP [8]、原型 [7]）。红色组件是 SimSiam 中省去的部分，下面讨论其关系。

与 SimCLR [8]的关系。SimCLR 依靠负样本（"不相似性"）避免坍塌。SimSiam 可视为"去掉负样本的 SimCLR"。

为更充分比较，我们给 SimCLR 加入预测 MLP h 与停止梯度。[7] 对自行复现的 SimCLR 消融如下：

SimCLR	使用预测器	使用预测器与停止梯度
66.5	66.4	66.0

停止梯度和额外预测器对 SimCLR 都不是必要的，也没有帮助。第5节分析认为，它们可能是另一底层优化问题的结果。该问题不同于对比学习，因此这些额外组件可能无益。

与 SwAV [7]的关系。概念上，SimSiam 类似于"去掉在线聚类的 SwAV"。我们通过重述 SwAV 的几个组件建立联系：（i）共享的原型层可并入孪生编码器；（ii）文献 [7]在梯度传播之外归一化原型权重，我们改为完全通过梯度计算实现 [33]。[8]（iii）SwAV 的相似性函数是交叉熵。经过这些抽象，高度简化的 SwAV 如图3所示。

SwAV 在目标分支应用 Sinkhorn-Knopp（SK）变换 [10]，同时也进行对称化 [7]。SK 变换源自在线聚类 [7]，是在均衡划分约束下对当前批次聚类的结果。均衡划分可避免坍塌。本方法不使用该变换。

我们研究预测 MLP h 与停止梯度对 SwAV 的影响。SwAV 在 SK 变换处停止梯度，因此通过移除该操作做消融。下面比较自行复现的 SwAV：

SwAV	w/ predictor	remove stop-grad
66.5	65.2	NaN

加入预测器也无益。去掉停止梯度、进行端到端训练会发散。作为聚类方法，SwAV 内在地具有

[7] 一侧添加预测器，另一侧停止梯度，再通过交换进行对称化。[8] 该修改的结果与原 SwAV 相近，但允许消融中进行端到端传播。

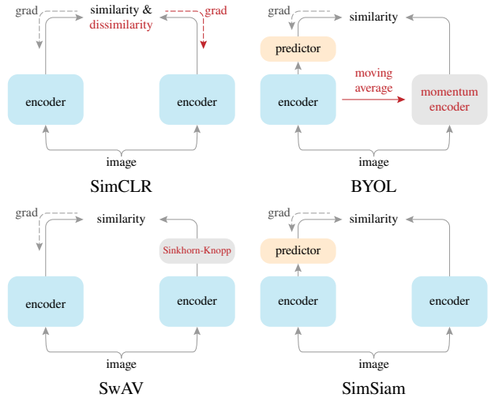


图3：孪生架构比较。编码器包含两个分支可共享的全部层。虚线表示梯度传播。BYOL、SwAV、SimSiam 中，没有虚线表示停止梯度；为简洁起见，不画出其对称化。红色组件为 SimSiam 中省去的部分。

交替形式 [7]。这可能解释为何 SwAV 的停止梯度不应被移除。

与 BYOL [15]的关系。尽管实现有诸多差异，本方法可视为"去掉动量编码器的 BYOL"。动量编码器可能提高准确率（表4），但不是避免坍塌的必要条件。基于第5节假设，η 子问题（8）可以用其他优化器求解，如基于梯度的方法，从而使 η的更新在时间上更平滑。虽然没有直接联系，动量编码器也产生更平滑的η。我们认为，求解式（8）的其他优化器同样合理，可作为未来研究问题。

7 结论

我们探索了设计简单的孪生网络。极简方法仍具有竞争力，说明近期方法的孪生形式可能是有效性的核心原因。孪生网络天然且有效地建模不变性，而不变性是表征学习的重点。希望本研究使社区关注孪生网络在表征学习中的基础作用。

参考文献

[1] Yuki Markus Asano, Christian Rupprecht, and Andrea Vedaldi. Self-labelling via simultaneous clustering and rep- resentation learning. arXiv:1911.05371, 2019.

[2] Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. *arXiv:1906.00910*, 2019.

[3] Luca Bertinetto, Jack Valmadre, Joao F Henriques, Andrea Vedaldi, and Philip HS Torr. Fully-convolutional Siamese networks for object tracking. In *ECCV*, 2016.

[4] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. Signature verification using a "Siamese" time delay neural network. In *NeurIPS*, 1994.

[5] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *ECCV*, 2018.

[6] Mathilde Caron, Piotr Bojanowski, Julien Mairal, and Armand Joulin. Unsupervised pre-training of image features on non-curated data. In *ICCV*, 2019.

[7] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Unsupervised learning of visual features by contrasting cluster assignments. *arXiv:2006.09882*, 2020.

[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. *arXiv:2002.05709*, 2020.

[9] Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. *arXiv:2003.04297*, 2020.

[10] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *NeurIPS*, 2013.

[11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[12] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The PASCAL Visual Object Classes (VOC) Challenge. *IJCV*, 2010.

[13] Ross Girshick, Ilija Radosavovic, Georgia Gkioxari, Piotr Dollár, and Kaiming He. Detectron, 2018.

[14] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*, 2017.

[15] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. *arXiv:2006.07733v1*, 2020.

[16] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *CVPR*, 2006.

[17] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. *arXiv:1911.05722*, 2019.

[18] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask R-CNN. In *ICCV*, 2017.

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[20] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler, and Yoshua Bengio. Learning deep representations by mutual information estimation and maximization. In *ICLR*, 2019.

[21] Olivier J. Hénaff, Aravind Srinivas, Jeffrey De Fauw, Ali Razavi, Carl Doersch, S. M. Ali Eslami, and Aaron van den Oord. Data-efficient image recognition with contrastive predictive coding. *arXiv:1905.09272v2*, 2019.

[22] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[23] Gregory Koch, Richard Zemel, and Ruslan Salakhutdinov. Siamese neural networks for one-shot image recognition. In *ICML deep learning workshop*, 2015.

[24] Alex Krizhevsky. Learning multiple layers of features from tiny images. *Tech Report*, 2009.

[25] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.

[26] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.

[27] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[28] James MacQueen et al. Some methods for classification and analysis of multivariate observations. 1967.

[29] Ishan Misra and Laurens van der Maaten. Self-supervised learning of pretext-invariant representations. *arXiv:1912.01991*, 2019.

[30] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv:1807.03748*, 2018.

[31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[32] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.

[33] Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *NeurIPS*, 2016.

[34] Yaniv Taigman, Ming Yang, MarcAurelio Ranzato, and Lior Wolf. DeepFace: Closing the gap to human-level performance in face verification. In *CVPR*, 2014.

[35] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. *arXiv:1906.05849*, 2019.

[36] Zhirong Wu, Yuanjun Xiong, Stella Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In *CVPR*, 2018.

[37] Mang Ye, Xu Zhang, Pong C Yuen, and Shih-Fu Chang. Unsupervised embedding learning via invariant and spreading instance feature. In *CVPR*, 2019.

[38] Yang You, Igor Gitman, and Boris Ginsburg. Large batch training of convolutional networks. *arXiv:1708.03888*, 2017.

[2] Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. arXiv:1906.00910, 2019.[3] Luca Bertinetto, Jack Valmadre, Joao F Henriques, Andrea Vedaldi, and Philip HS Torr. Fully-convolutional Siamese networks for object tracking. In ECCV, 2016.[4] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard S¨ackinger, and Roopak Shah. Signature verification using a "Siamese" time delay neural network. In NeurIPS, 1994.[5] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In ECCV, 2018.[6] Mathilde Caron, Piotr Bojanowski, Julien Mairal, and Ar- mand Joulin. Unsupervised pre-training of image features on non-curated data. In ICCV, 2019.[7] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Pi- otr Bojanowski, and Armand Joulin. Unsupervised learn- ing of visual features by contrasting cluster assignments. arXiv:2006.09882, 2020.
[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Ge- offrey Hinton. A simple framework for contrastive learning of visual representations. arXiv:2002.05709, 2020.
[9] Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. arXiv:2003.04297, 2020.[10] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In NeurIPS, 2013.[11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In CVPR, 2009.
[12] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The PASCAL Visual Object Classes (VOC) Challenge. IJCV, 2010.[13] Ross Girshick, Ilija Radosavovic, Georgia Gkioxari, Piotr Doll´ar, and Kaiming He. Detectron, 2018.[14] Priya Goyal, Piotr Doll´ar, Ross Girshick, Pieter Noord- huis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. arXiv:1706.02677, 2017.[15] Jean-Bastien Grill, Florian Strub, Florent Altch´e, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Do- ersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Moham- mad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, R´ emi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. arXiv:2006 1, 2020 07733. v .[16] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimension- ality reduction by learning an invariant mapping. In CVPR, 2006.[17] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual rep- resentation learning. arXiv:1911.05722, 2019.[18] Kaiming He, Georgia Gkioxari, Piotr Doll´ar, and Ross Gir- shick. Mask R-CNN. In ICCV, 2017.[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In CVPR, 2016.
[20] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler, and Yoshua Bengio. Learn- ing deep representations by mutual information estimation and maximization. In ICLR, 2019.[21] Olivier J. H´enaff, Aravind Srinivas, Jeffrey De Fauw, Ali Razavi, Carl Doersch, S. M. Ali Eslami, and Aaron van den Oord. Data-efficient image recognition with contrastive pre- dictive coding. arXiv:1905.09272v2, 2019. [22] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal co- variate shift. In ICML, 2015.[23] Gregory Koch, Richard Zemel, and Ruslan Salakhutdinov. Siamese neural networks for one-shot image recognition. In ICML deep learning workshop, 2015. [24] Alex Krizhevsky. Learning multiple layers of features from tiny images. Tech Report, 2009.[25]Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwrit- ten zip code recognition. Neural computation, 1989.[26] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Doll´ar, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In ECCV. 2014.[27] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradi- ent descent with warm restarts. In ICLR, 2017.[28] James MacQueen et al. Some methods for classification and analysis of multivariate observations. 1967. [29] Ishan Misra and Laurens van der Maaten. Self- supervised learning of pretext-invariant representations. arXiv:1912.01991, 2019.[30] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Rep- resentation learning with contrastive predictive coding. arXiv:1807.03748, 2018.[31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In NeurIPS, 2019.[32] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with re- gion proposal networks. In NeurIPS, 2015. [33] Tim Salimans and Diederik P Kingma. Weight normaliza- tion: A simple reparameterization to accelerate training of deep neural networks. In NeurIPS, 2016.[34] Yaniv Taigman, Ming Yang, MarcAurelio Ranzato, and Lior Wolf. DeepFace: Closing the gap to human-level perfor- mance in face verification. In CVPR, 2014.[35] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Con- trastive multiview coding. arXiv:1906.05849, 2019.
[36] Zhirong Wu, Yuanjun Xiong, Stella Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In CVPR, 2018.[37] Mang Ye, Xu Zhang, Pong C Yuen, and Shih-Fu Chang. Unsupervised embedding learning via invariant and spread- ing instance feature. In CVPR, 2019.
[38] Yang You, Igor Gitman, and Boris Ginsburg. Large batch training of convolutional networks. arXiv:1708.03888, 2017.

A. Implementation Details

Unsupervised pre-training. Our implementation follows the practice of existing works [36, 17, 8, 9, 15].

Data augmentation. We describe data augmentation using the PyTorch [31] notations. Geometric augmentation is `RandomResizedCrop` with scale in $[0.2, 1.0]$ [36] and `RandomHorizontalFlip`. Color augmentation is `ColorJitter` with {brightness, contrast, saturation, hue} strength of {0.4, 0.4, 0.4, 0.1} with an applying probability of 0.8, and `RandomGrayscale` with an applying probability of 0.2. Blurring augmentation [8] has a Gaussian kernel with std in $[0.1, 2.0]$.

Initialization. The convolution and fc layers follow the default PyTorch initializers. Note that by default PyTorch initializes fc layers' weight and bias by a uniform distribution $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k=\frac{1}{\text{in_channels}}$. Models with substantially different fc initializers (*e.g.*, a fixed std of 0.01) may not converge. Moreover, similar to the implementation of [8], we initialize the scale parameters as 0 [14] in the last BN layer for every residual block.

Weight decay. We use a weight decay of 0.0001 for all parameter layers, including the BN scales and biases, in the SGD optimizer. This is in contrast to the implementation of [8, 15] that excludes BN scales and biases from weight decay in their LARS optimizer.

Linear evaluation. Given the pre-trained network, we train a supervised linear classifier on frozen features, which are from ResNet's global average pooling layer (pool_5). The linear classifier training uses base $lr = 0.02$ with a cosine decay schedule for 90 epochs, weight decay $= 0$, momentum$=0.9$, batch size$=4096$ with a LARS optimizer [38]. We have also tried the SGD optimizer following [17] with base $lr = 30.0$, weight decay $= 0$, momentum $= 0.9$, and batch size$=256$, which gives ~1% lower accuracy. After training the linear classifier, we evaluate it on the center 224×224 crop in the validation set.

B. Additional Ablations on ImageNet

The following table reports the SimSiam results *vs*. the output dimension d:

output d	256	512	1024	2048
acc. (%)	65.3	67.2	67.5	68.1

It benefits from a larger d and gets saturated at $d = 2048$. This is unlike existing methods [36, 17, 8, 15] whose accuracy is saturated when d is 256 or 512.

In this table, the prediction MLP's hidden layer dimension is always 1/4 of the output dimension. We find that this bottleneck structure is more robust. If we set the hidden dimension to be equal to the output dimension, the training can be less stable or fail in some variants of our exploration. We hypothesize that this bottleneck structure, which behaves like an auto-encoder, can force the predictor to digest the information. We recommend to use this bottleneck structure for our method.

	SimCLR			MoCo v2		BYOL			SwAV
epoch	200	800	1000	200	800	300	800	1000	400
origin	66.6	68.3	69.3	67.5	71.1	72.5	-	74.3	70.1
repro.	68.3	70.4	-	69.9	72.2	72.4	74.3	-	70.7

Table C.1. **Our reproduction *vs*. original papers' results**. All are based on ResNet-50 pre-trained with two 224×224 crops.

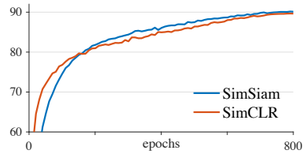
C. Reproducing Related Methods

Our comparison in Table 4 is based on our reproduction of the related methods. We re-implement the related methods as faithfully as possible following each individual paper. In addition, we are able to improve SimCLR, MoCo v2, and SwAV by small and straightforward modifications: specifically, we use 3 layers in the projection MLP in SimCLR and SwAV (*vs*. originally 2), and use symmetrized loss for MoCo v2 (*vs*. originally asymmetric). Table C.1 compares our reproduction of these methods with the original papers' results (if available). Our reproduction has *better* results for SimCLR, MoCo v2, and SwAV (denoted as "+" in Table 4), and has at least comparable results for BYOL.

D. CIFAR Experiments

We have observed similar behaviors of SimSiam in the CIFAR-10 dataset [24]. The implementation is similar to that in ImageNet. We use SGD with base $lr = 0.03$ and a cosine decay schedule for 800 epochs, weight decay $=$ 0.0005, momentum $= 0.9$, and batch size $= 512$. The input image size is 32×32. We do not use blur augmentation. The backbone is the CIFAR variant of ResNet-18 [19], followed by a 2-layer projection MLP. The outputs are 2048-d.

Figure D.1 shows the kNN classification accuracy (left) and the linear evaluation (right). Similar to the ImageNet observations, SimSiam achieves a reasonable result and does not collapse. We compare with SimCLR [8] trained with the same setting. Interestingly, the training curves are similar between SimSiam and SimCLR. SimSiam is slightly better by 0.7% under this setting.



	acc. (%)
SimCLR	91.1
SimSiam	91.8

Figure D.1. **CIFAR-10 experiments**. Left: validation accuracy of kNN classification as a monitor during pre-training. Right: linear evaluation accuracy. The backbone is ResNet-18.

A 实现细节

无监督预训练。实现遵循现有研究 [36, 17, 8, 9, 15]的惯例。

数据增强。我们用 PyTorch [31] 的记法描述。几何增强为缩放范围 $[0.2, 1.0]$[36]的 RandomResizedCrop 及 RandomHorizontalFlip。颜色增强为 ColorJitter，{亮度、对比度、饱和度、色相} 强度分别为 {0.4、0.4、0.4、0.1}，应用概率0.8；RandomGrayscale 应用概率0.2。模糊增强 [8] 采用标准差位于 $[0.1, 2.0]$的高斯核。

初始化。卷积和全连接层使用 PyTorch 默认初始化器。默认情况下，全连接层的权重与偏置按均匀分布 $\mathcal{U}$（$-\sqrt{k}$，$\sqrt{k}$）初始化，其中 $k=\frac{1}{\text{输入通道数}}$。显著不同的全连接初始化（如固定标准差0.01）可能导致不收敛。此外，类似于 [8], 的实现，每个残差块的最后一个 BN 层将缩放参数初始化为 0 [14]。

权重衰减。SGD 中，所有参数层均使用0.0001的权重衰减，包括 BN 缩放与偏置。这与 [8, 15] 的实现不同，后者在 LARS 中不对 BN 缩放和偏置做权重衰减。

线性评估。给定预训练网络，在冻结特征上训练有监督线性分类器，特征来自 ResNet 全局平均池化层（pool_5）。线性分类器采用基础 $lr = 0.02$ 、90轮余弦衰减日程、权重衰减 $= 0$、动量$=0.9$、批量大小$=4096$，使用 LARS 优化器[38]。我们也按照 [17]尝试了 SGD，基础 $lr = 30.0$、权重衰减 $= 0$、动量 $= 0.9$、批量$=256$；准确率低 ~1个百分点。训练后，在验证集的224×224中心裁剪上评估。

B 更多 ImageNet 消融

下表报告 SimSiam 结果与输出维数 d的关系：

输出 d	256	512	1024	2048
准确率（%）	65.3	67.2	67.5	68.1

它受益于更大的 d，到 $d = 2048$时饱和。已有方法 [36, 17, 8, 15] 则不同，其准确率在 d 为256或512时就饱和。

表中，预测 MLP 隐藏层维数始终为输出维数的 1/4。瓶颈结构更稳健。若隐藏维数与输出维数相同，训练

可能更不稳定，或在某些变体中失败。我们假设，这种瓶颈结构

	SimCLR			MoCo v2		BYOL			SwAV
轮次	200	800	1000	200	800	300	800	1000	400
原论文	66.6	68.3	69.3	67.5	71.1	72.5	-	74.3	70.1
复现	68.3	70.4	-	69.9	72.2	72.4	74.3	-	70.7

表 C.1：我们的复现与原论文结果比较。全部采用 ResNet-50，以两个 224×224裁剪预训练。

类似于自编码器，可迫使预测器消化信息。我们建议本方法使用这种瓶颈结构。

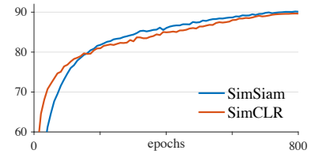
C 复现相关方法

表4的比较基于我们对相关方法的复现。我们尽可能忠实地遵循各原始论文重新实现。此外，通过简单、小幅修改改进了 SimCLR、MoCo v2 和 SwAV：SimCLR 与 SwAV 的投影 MLP 使用3层（原来为2层）；MoCo v2 使用对称损失（原来不对称）。表 C.1 将复现结果与原论文结果比较（若已提供）。SimCLR、MoCo v2 和 SwAV 的复现结果更好（表4中标为"+"），BYOL 至少达到相近结果。

D CIFAR 实验

在 CIFAR-10 [24]上，我们也观察到 SimSiam 的类似行为。实现与 ImageNet 相近：采用 SGD，基础 $lr = 0.03$ 、800轮余弦衰减、权重衰减 =0.0005、动量$=0.9$、批量$=512$。输入尺寸为 32×32，不使用模糊增强。骨干为 CIFAR 版 ResNet-18 [19],，后接2层投影 MLP，输出为2048维。

图 D.1 展示 kNN 分类准确率（左）及线性评估（右）。与 ImageNet 的观察相似，SimSiam 得到合理结果，不发生坍塌。我们与相同设置下训练的 SimCLR [8] 比较。两者训练曲线相近；该设置下，SimSiam 略高0.7个百分点。



	acc. (%)
SimCLR	91.1
SimSiam	91.8

图 D.1：CIFAR-10 实验。左：预训练时用作监测器的 kNN 验证准确率；右：线性评估准确率。骨干为 ResNet-18。